Python में File Handling

1. Introduction / परिचय

What is File Handling? / File Handling क्या है?

English: File handling is the process of creating, reading, updating, and deleting files using Python.

Hindi: File handling Python में files को create, read, update और delete करने की प्रक्रिया है।

Why File Handling? / File Handling क्यों?

- Data को permanently store करने के लिए
- Large data को process करने के लिए
- Configuration files को manage करने के लिए
- Data को programs के बीच share करने के लिए

---

2. File Opening Modes / File खोलने के Modes

Mode	Name	Description (English)	विवरण (Hindi)
`'r'`	Read	Read only (default)	केवल पढ़ने के लिए (default)
`'w'`	Write	Write (overwrites)	लिखने के लिए (पुराना data delete हो जाता है)
`'a'`	Append	Write at end	अंत में लिखने के लिए
`'x'`	Exclusive	Create new file	नई file बनाने के लिए (पहले से exist तो error)
`'r+'`	Read/Write	Read and write	पढ़ने और लिखने दोनों के लिए
`'w+'`	Write/Read	Write and read (overwrites)	लिखने और पढ़ने के लिए (overwrite)
`'a+'`	Append/Read	Append and read	Append और read के लिए
`'rb'`	Read Binary	Read binary files	Binary files पढ़ने के लिए
`'wb'`	Write Binary	Write binary files	Binary files लिखने के लिए

---

3. Basic File Operations / बुनियादी File Operations

Opening a File / File खोलना

```
# Method 1: Basic open
# English
file = open("data.txt", "r")
# Hindi
file = open("data.txt", "r")  # file खोलें

# Method 2: With statement (recommended)
# English
with open("data.txt", "r") as file:
    content = file.read()
```

Closing a File / File बंद करना

```
# Manual closing
# English
file = open("data.txt", "r")
# ... operations ...
file.close()


# With statement (auto-closes)
# Automatically close होती है
with open("data.txt", "r") as file:
    content = file.read()
```

---

4. Reading Files / Files पढ़ना

Method 1: read() - पूरी File पढ़ना

```
# English Example
with open("data.txt", "r") as file:
    content = file.read()  # Read entire file
    print(content)


# Read specific number of characters
# निश्चित संख्या में characters पढ़ें
with open("data.txt", "r") as file:
    content = file.read(10)  # First 10 characters / पहले 10 characters
    print(content)
```

Method 2: readline() - एक Line पढ़ना

```
# English Example
with open("data.txt", "r") as file:
    line1 = file.readline()  # Read first line
    line2 = file.readline()  # Read second line
    print(line1)
    print(line2)
```

Method 3: readlines() - सभी Lines को List में

```
# English Example
with open("data.txt", "r") as file:
    lines = file.readlines()  # Returns list of lines
    for line in lines:
        print(line.strip())  # strip() removes newline
```

Method 4: Loop के साथ पढ़ना (Best Practice)

```
# English Example - Memory efficient
with open("data.txt", "r") as file:
    for line in file:  # Reads line by line
        print(line.strip())
```

---

5. Writing to Files / Files में लिखना

Method 1: write() - String लिखना

```
# English Example - Overwrite mode
with open("output.txt", "w") as file:
    file.write("Hello, World!\n")
    file.write("This is a new line.\n")

# Append mode - Add to existing content
# Append mode - मौजूदा content में जोड़ें
with open("output.txt", "a") as file:
    file.write("Additional line\n")
    file.write("अतिरिक्त line\n")
```

Method 2: writelines() - Multiple Lines लिखना

```
# English Example
lines = ["Line 1\n", "Line 2\n", "Line 3\n"]
with open("output.txt", "w") as file:
    file.writelines(lines)
```

---

6. File Positions / File Positions

tell() - Current Position बताना

```
with open("data.txt", "r") as file:
    print(file.tell())  # Current position: 0
    file.read(5)
    print(file.tell())  # Current position: 5
```

seek() - Position बदलना

```
with open("data.txt", "r") as file:
    file.seek(10)  # Move to position 10
    content = file.read(5)  # Read 5 characters from position 10
    print(content)
```

---

7. File Operations with os Module / os Module के साथ File Operations

8. Directory Operations / Directory Operations

Creating Directory / Directory बनाना

```
import os

# English Example
os.mkdir("new_folder")  # Single directory
os.makedirs("parent/child/grandchild")  # Multiple directories
```

Listing Directory Contents / Directory की Contents देखना

```
import os

# English Example
files = os.listdir(".")  # Current directory
print("Files in current directory:")
for file in files:
    print(file)
```

Changing Directory / Directory बदलना

```
import os

# English Example
print("Current directory:", os.getcwd())
os.chdir("new_folder")
print("New directory:", os.getcwd())
```

Removing Directory / Directory हटाना

```
import os

# English Example
os.rmdir("empty_folder")  # Remove empty directory
import shutil
shutil.rmtree("folder_with_files")  # Remove directory with contents
```

---

9. Binary Files / Binary Files

Reading Binary Files / Binary Files पढ़ना

```
# English Example - Reading images, executables, etc.
with open("image.jpg", "rb") as file:
    binary_data = file.read()
    print(f"File size: {len(binary_data)} bytes")
```

Writing Binary Files / Binary Files लिखना

```
# English Example - Copying a file
with open("source.jpg", "rb") as source:
    with open("destination.jpg", "wb") as dest:
        dest.write(source.read())
```

---

10. CSV Files / CSV Files

Reading CSV Files / CSV Files पढ़ना

```
import csv

with open("data.csv", "r") as file:
    csv_reader = csv.reader(file)
    for row in csv_reader:
        print(row)  # Each row is a list

# Using DictReader
with open("data.csv", "r") as file:
    csv_reader = csv.DictReader(file)
    for row in csv_reader:
        print(row)  # Each row is a dictionary
```

Writing CSV Files / CSV Files लिखना

```
import csv

# English Example
data = [
    ["Name", "Age", "City"],
    ["Rahul", 25, "Delhi"],
    ["Priya", 22, "Mumbai"]
]

with open("output.csv", "w", newline='') as file:
    csv_writer = csv.writer(file)
    csv_writer.writerows(data)
```

---

11. JSON Files / JSON Files

Reading JSON Files / JSON Files पढ़ना

```
import json

# English Example
with open("data.json", "r") as file:
    data = json.load(file)
    print(data)
    print(data["name"])  # Access specific key
```

Writing JSON Files / JSON Files लिखना

```
import json

# English Example
data = {
    "name": "Rahul",
    "age": 25,
    "city": "Delhi",
    "skills": ["Python", "Java", "SQL"]
}

with open("output.json", "w") as file:
    json.dump(data, file, indent=4)
```

---

12. Exception Handling with Files / Files के साथ Exception Handling

```
# English Example
try:
    with open("data.txt", "r") as file:
        content = file.read()
        print(content)
except FileNotFoundError:
    print("Error: File not found")
except PermissionError:
    print("Error: Permission denied")
except Exception as e:
    print(f"Error: {e}")
```

---

13. Practical Examples / व्यावहारिक उदाहरण

Example 1: Word Counter / शब्द गणक

```
# English Example
def count_words(filename):
    try:
        with open(filename, "r") as file:
            content = file.read()
            words = content.split()
            return len(words)
    except FileNotFoundError:
        return "File not found"

result = count_words("document.txt")
print(f"Total words: {result}")
```

Example 2: File Copy / File Copy करना

```
# English Example
def copy_file(source, destination):
    try:
        with open(source, "r") as src:
            with open(destination, "w") as dest:
                dest.write(src.read())
        return "File copied successfully"
    except FileNotFoundError:
        return "Source file not found"
    except Exception as e:
        return f"Error: {e}"

result = copy_file("input.txt", "output.txt")
print(result)
```

Example 3: Student Data Manager / ছাত্র Data Manager

```
import csv

# English Example
def add_student(name, age, grade):
    with open("students.csv", "a", newline='') as file:
        writer = csv.writer(file)
        writer.writerow([name, age, grade])
    print("Student added successfully")

def view_students():
    try:
        with open("students.csv", "r") as file:
            reader = csv.reader(file)
            print("\nStudent List:")
            print("-" * 40)
            for row in reader:
                print(f"Name: {row[0]}, Age: {row[1]}, Grade: {row[2]}")
    except FileNotFoundError:
        print("No student records found")
```

Example 4: Log File Manager / Log File Manager

```
from datetime import datetime

# English Example
def write_log(message, level="INFO"):
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    log_entry = f"[{timestamp}] [{level}] {message}\n"

    with open("app.log", "a") as file:
        file.write(log_entry)

def read_logs():
    try:
        with open("app.log", "r") as file:
            logs = file.readlines()
            return logs
    except FileNotFoundError:
        return []

# Usage
write_log("Application started", "INFO")
write_log("Error occurred", "ERROR")
logs = read_logs()
for log in logs:
    print(log.strip())
```

---

14. Best Practices / सर्वोत्तम Practices

English:

1. **Always use `with` statement** - Automatically closes files
2. **Handle exceptions** - Use try-except blocks
3. **Use appropriate modes** - Choose correct read/write mode
4. **Close files explicitly** - If not using `with`
5. **Use encoding** - Specify encoding='utf-8' for text files
6. **Check file existence** - Before operations
7. **Use relative paths** - When possible
8. **Avoid hardcoding paths** - Use os.path.join()
9. **Buffer large files** - Read in chunks for large files
10. **Clean up temporary files** - Delete when no longer needed

Hindi:

1. **हमेशा `with` statement use करें** - Files automatically close होती हैं
2. **Exceptions handle करें** - Try-except blocks use करें
3. **उचित modes use करें** - सही read/write mode चुनें
4. **Files को explicitly close करें** - अगर `with` use नहीं कर रहे
5. **Encoding use करें** - Text files के लिए encoding='utf-8' specify करें
6. **File existence check करें** - Operations से पहले
7. **Relative paths use करें** - जब संभव हो
8. **Paths hardcode न करें** - os.path.join() use करें
9. **Large files को buffer करें** - Chunks में पढ़ें

10.**Temporary files clean करें** - जरूरत न होने पर delete करें

---

15. Common Errors and Solutions / सामान्य Errors और समाधान

Error 1: FileNotFoundError

```
# Problem / समस्या
file = open("nonexistent.txt", "r")  # File doesn't exist

# Solution / समाधान
try:
    with open("nonexistent.txt", "r") as file:
        content = file.read()
except FileNotFoundError:
    print("File not found / File नहीं मिली")
```

Error 2: PermissionError

```
# Problem / समस्या
file = open("protected.txt", "w")  # No write permission

# Solution / समाधान
try:
    with open("protected.txt", "w") as file:
        file.write("data")
except PermissionError:
```

```
    print("Permission denied / Permission नहीं है")
```

Error 3: UnicodeDecodeError

```
# Problem / समस्या
with open("hindi.txt", "r") as file:  # Missing encoding
    content = file.read()

# Solution / समाधान
with open("hindi.txt", "r", encoding='utf-8') as file:
    content = file.read()
```

---

16. File Handling Methods Summary / Methods का सारांश

File Object Methods

Method	Description (English)	विवरण (Hindi)
`read()`	Read entire file	पूरी file पढ़ें
`readline()`	Read one line	एक line पढ़ें
`readlines()`	Read all lines as list	सभी lines list में
`write()`	Write string to file	String file में लिखें
`writelines()`	Write list of strings	Strings की list लिखें
`tell()`	Get current position	वर्तमान position
`seek()`	Change position	Position बदलें
`close()`	Close file	File बंद करें
`flush()`	Flush buffer	Buffer flush करें

---

17. Summary / सारांश

English:

- Files can be opened in different modes (read, write, append)
- Always use `with` statement for automatic file closing
- Handle exceptions to prevent crashes
- Use appropriate encoding for text files
- CSV and JSON modules for structured data
- os module for file and directory operations

Hindi:

- Files को विभिन्न modes में खोला जा सकता है (read, write, append)
- Automatic file closing के लिए `with` statement use करें
- Crashes से बचने के लिए exceptions handle करें
- Text files के लिए उचित encoding use करें
- Structured data के लिए CSV और JSON modules
- File और directory operations के लिए os module

Quick Reference Card / त्वरित संदर्भ

```
# File Opening / File खोलना
with open("file.txt", "r") as f:
    content = f.read()

# File Writing / File लिखना
with open("file.txt", "w") as f:
    f.write("Hello World")

# File Appending / File में जोड़ना
with open("file.txt", "a") as f:
    f.write("New line")

# CSV Reading / CSV पढ़ना
import csv
with open("data.csv", "r") as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)

# JSON Reading / JSON पढ़ना
import json
with open("data.json", "r") as f:
    data = json.load(f)
```